



**TRƯỜNG ĐẠI HỌC
CÔNG NGHỆ ĐÔNG Á**
EAST ASIA UNIVERSITY OF TECHNOLOGY

International Training & Cooperation Institute

Introduction to Blockchain Technology

Lesson 10 – Handling Ether in Smart Contracts



Handling Ether in Smart Contracts



- 1. Ether in Ethereum**
- 2. Contract ABI**
- 3. Payable Function**
- 4. Ether Storage & Transfer Methods**
- 5. Withdraw From Ether Contract**
- 6. Security Considerations When Handling Ether**
- 7. Exercises**

1. Ether in Ethereum

- ❖ Ether (ETH) is Ethereum's native currency
- ❖ Used to:
 - Pay transaction fees (gas)
 - Transfer value
- ❖ Smart contracts can:
 - Receive Ether
 - Hold Ether
 - Send Ether
- ❖ Ether belongs to an **address**
- ❖ Smart contracts have their own address

1. Ether in Ethereum

Ether Unit

- ❖ Wei (smallest unit)

 - ❖ 1 Ether = 10^{18} (1,000,000,000,000,000,000) Wei

- ❖ Gwei (Giga Wei)

 - ❖ 1 Gwei = 10^9 (1,000,000,000) Wei

- ❖ Solidity uses Wei internally

2. Contract ABI

Application Binary Interface (ABI)

- ❖ The instruction manual that tells external applications how to talk to a smart contract.
- ❖ Smart contracts are deployed as **bytecode** on the blockchain
- ❖ ABI translates:
 - Function names
 - Input/output types
 - Parameter order

=> Allow interactions with any deployed contract using the **contract address + ABI**

```
pragma solidity ^0.8.0;

contract Hello {
    string public message;

    constructor(string memory _msg) {
        message = _msg;
    }

    function setMessage(string memory _msg)
    public {
        message = _msg;
    }
}
```



```
[
  {
    "inputs": [
      {
        "internalType": "string",
        "name": "_msg",
        "type": "string"
      }
    ],
    "stateMutability": "nonpayable",
    "type": "constructor"
  },
  {
    "inputs": [],
    "name": "message",
    "outputs": [
      {
        "internalType": "string",
        "name": "",
        "type": "string"
      }
    ],
    "stateMutability": "view",
    "type": "function"
  },
  {
    "inputs": [
      {
        "internalType": "string",
        "name": "_msg",
        "type": "string"
      }
    ],
    "name": "setMessage",
    "outputs": [],
    "stateMutability": "nonpayable",
    "type": "function"
  }
]
```

ABI is a JSON array

2. Contract ABI



Tip: Getting Contract ABIs from explorer page

- ❖ To interact with or audit an existing contract on the blockchain, you do not need the original Solidity (.sol) source code file. You only need two elements:
 - The contract's On-Chain Address
 - The contract's Application Binary Interface (ABI)

Step-by-Step ABI Retrieval on Blockscout:

- 1) Go to the Sepolia Blockscout explorer:
<https://eth-sepolia.blockscout.com/>
- 2) Search for your target Contract Address in the search bar.
- 3) Click on the **Contract** tab.
- 4) Scroll down to the **ABI** section and click the **Copy** button.

Note: After deploying a contract on Remix, you can also extract the contract ABI in the **Deploy & Run Transactions** tab

The screenshot displays the Sepolia Blockscout explorer interface. At the top, the URL is https://eth-sepolia.blockscout.com/address/0x92fed41e9400d5AffA36eE3C387FAfFaf457602B?tab=contract_abi. The page title is "Contract details" for "MessageApp". Below the title, the contract address "0x92fed41e9400d5AffA36eE3C387FAfFaf457602B" is shown with icons for details, transactions, and logs. A navigation bar includes tabs for "Details", "Contract" (selected), "Transactions 3", "Token transfers 0", "Tokens 0", "Internal txns 0", "Coin balance history", and "Logs 2". Below this, there are buttons for "Code", "Read/Write contract", and "Custom ABI". A green banner states "Contract source code verified (exact match)". A table of contract metadata follows, including Contract name (MessageApp), Compiler version (v0.8.31+commit.fd3a2265), EVM version (default), Optimization enabled (false), Verified at (Jan 26, 2026 8:05:50), and Contract file path (msg.sol). At the bottom, there are tabs for "Code", "Compiler", "ABI" (selected), and "Bytecode". The "Contract ABI" section shows a JSON array of function signatures, with the first function being `_msg(string)`.

```
1  [
2  {
3    "inputs": [
4      {
5        "internalType": "string",
6        "name": "_msg",
7        "type": "string"
8      }
9    ],
10   "stateMutability": "nonpayable",
```

2. Contract ABI



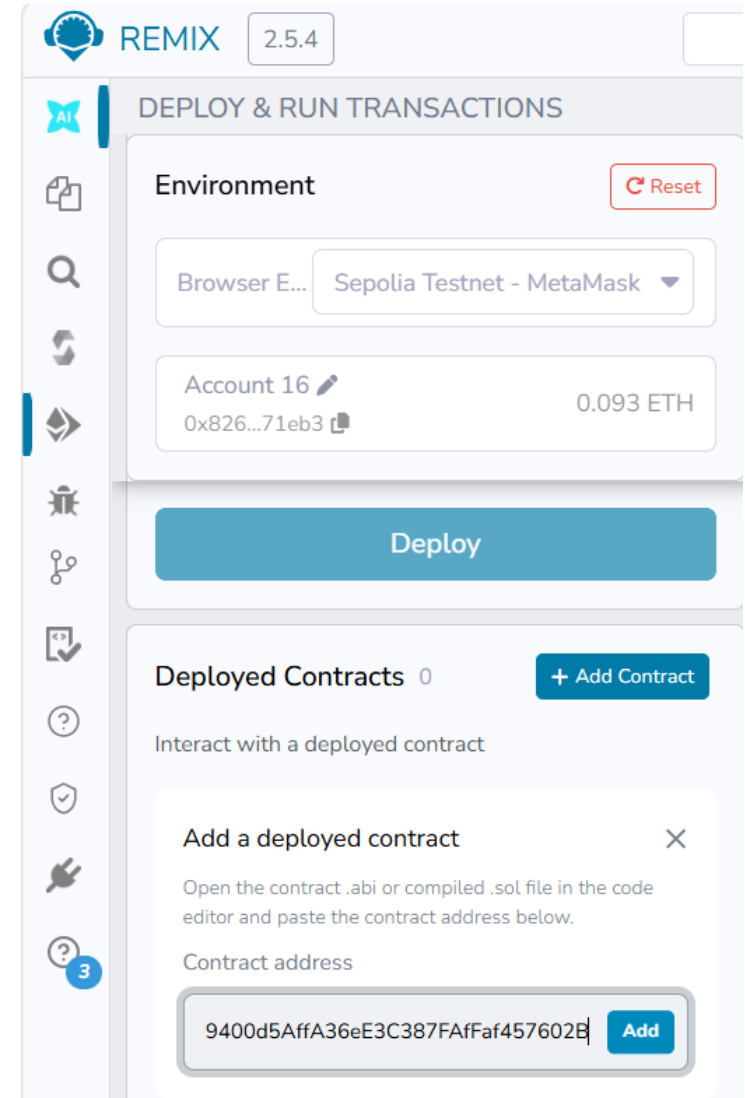
Interacting with Deployed Contracts in Remix

Interact with a contract already live on the blockchain without compiling any source code yourself

Step-by-Step in Remix:

- 1) Open Remix IDE: remix.ethereum.org.
- 2) Create a new file, e.g., named MyContract.abi.
- 3) Paste the contract's ABI JSON array into the file.
- 4) Keep the **.abi** file open and active in the editor.
- 5) Open Deploy & Run Transactions.
- 6) Set Environment to Browser Extension and connect MetaMask to the correct network, e.g. Sepolia.
- 7) Under Deployed Contracts, click Add Contract.
- 8) Enter the deployed contract's address.
- 9) Click Add

=> Remix adds the contract under Deployed Contracts.
Its functions appear based on the ABI.



3. Payable Function

payable: allows the function to receive Ether

- ❖ Without **payable**, sending **Ether** will fail
- ❖ Ether is sent together with the transaction

```
function deposit() public payable {  
    uint amount = msg.value;  
}
```

- ❖ **msg.value** = amount of Ether sent
- ❖ Available only in payable functions
- ❖ Value is in Wei

- ❖ **Receiving Plain Ether:** Users can send ETH directly to the contract address.

```
receive() external payable { }
```

receive() is executed when Ether is sent directly to the contract address without calling a function.

4. Ether Storage & Transfer Methods

Contract Balance

- ❖ Returns Ether balance of the contract
- ❖ Read-only
- ❖ Does NOT cost gas when called as **view**

```
address(this).balance
```

4. Ether Storage & Transfer Methods

Example

What this contract does

- ❖ *Accepts Ether*
- ❖ *Stores Ether*
- ❖ *Shows total balance*

```
pragma solidity ^0.8.0;

contract EtherBox {
    function deposit() public payable {}

    function getBalance() public view returns (uint) {
        return address(this).balance;
    }
}
```

Questions:

- ❖ Does *deposit* cost gas?
- ❖ Does *getBalance* cost gas?
- ❖ Where is the Ether stored?

4. Ether Storage & Transfer Methods

- ❖ **transfer()** and **send()** fail if receiver needs more gas
- ❖ Many contracts today require more than 2300 gas
- ❖ **call()** is flexible and future-proof
- ❖ Official Ethereum docs recommend **call()**

Method	Gas Limit
transfer()	2300 gas
send()	2300 gas
call()	Adjustable

5. Withdraw From Ether Contract

Example

```
function withdraw() public {  
    require(msg.sender == owner, "Not owner");  
    uint256 balance = address(this).balance;  
    require(balance > 0, "No funds to withdraw");  
  
    (bool success, ) = payable(owner).call{value: balance}("");  
    require(success, "Transfer failed");  
}
```

What this function does

- ❖ *Only owner can withdraw*
- ❖ *Entire balance is transferred*
- ❖ *Ether leaves the contract*

6. Security Considerations When Handling Ether

❖ Checks–Effects–Interactions

- Check permissions & balance first
- Update state (if any)
- Send Ether last

❖ Always verify against **reentrancy** risk

❖ Push vs Pull

- Pushing Ether (owner withdraws all) can fail
- Safer pattern: users pull their own funds

❖ Ownership Risk

- If owner loses key → funds locked forever
- Real systems add ownership transfer

❖ Gas Risk

- Sending Ether to contracts may fail if they need more gas

7. Exercises

Ex 1: Build a simple wallet contract with **deposit/withdraw** function

- ❖ Anyone can send Ether
- ❖ Only owner can withdraw
- ❖ Anyone can check balance

Ex 2: Build a **StudentFund** contract

Create a contract that collects funds for student activities.

- ❖ Anyone can donate Ether
- ❖ Owner can withdraw all funds
- ❖ Anyone can see total balance

7. Exercises



```
pragma solidity ^0.8.0;
contract RejectEther {
    receive() external payable {
        revert("I do not accept Ether");
    }
}
```

RejectEther

Ex 3: Given the above smart contracts.

❖ Deploy **RejectEther**

❖ Deploy **BrokenWallet**, passing **RejectEther** address as owner

- Who can call `withdraw()`?
- Did the `withdraw()` transaction fail or succeed?
- How can we detect that the transfer failed?

```
pragma solidity ^0.8.0;

contract BrokenWallet {
    address public owner;

    constructor(address _owner) {
        owner = _owner;
    }

    function deposit() public payable {}

    function withdraw() public {
        require(msg.sender == owner, "Not owner");
        payable(owner).call{value: address(this).balance}("");
    }

    function getBalance() public view returns (uint) {
        return address(this).balance;
    }
}
```

BrokenWallet

7. Exercises

Ex 4: Build a **LockedWallet** contract, that adding the time lock feature to the Wallet contract.

- ❖ Ether can be deposited by anyone
- ❖ Owner cannot withdraw until a specific time
- ❖ Time is set at deployment